

CNN Exam Study Guide

Applied Deep Learning — Convolutional Neural Networks

Hof University · Prof. Dr. Lars Ackermann-Igl

This guide covers **every concept** from the CNN lecture slides in the order they were taught. Each question has three levels: (1) a plain-English explanation so you truly understand it, (2) a worked numeric example with real numbers, and (3) a highlighted **Exam-Ready Answer** box showing what to write under time pressure. The mock exam tasks have been analysed — all likely question types are covered.

Part	Topic	Key Points
1	Why CNNs? MLP Problems	MLP limits, curse of dimensionality, image challenges
2	Image Input as 3D Volume	Width × height × depth, RGB channels
3	Convolution Operation	Kernel, receptive field, feature map, math formula
4	Padding & Stride	Border pixels, step size, output-size formula
5	Parameter Sharing	Why few weights, multiple filters, feature levels
6	Pooling Layer	Max-pooling, benefits, formula
7	Flatten → FC → Softmax	Classification pipeline, softmax formula
8	Calculation Practice	Full worked exam question (padding+stride+ReLU)
9	Extra Exam Questions	Comparison table, CNN vs regression, ReLU
10	Cheat Sheet	Everything on one page

PART 1 — Why CNNs? The Problem with MLPs and Images

Q1. What real-world tasks involve image data, and what makes each different?

Before understanding CNNs, you need to understand **what problems they solve**. There are four main computer-vision tasks:

- **Classification** — Look at the whole image and assign it ONE label. Example: is this photo a 'cat', 'dog', or 'ship'? Output = one class name.
- **Detection** — Find WHERE objects are AND what they are. The output is a box drawn around each object plus a label and confidence score. Example: person:0.99 (with a box around them).
- **Retrieval** — Given one image, find other images in a large database that look similar. Used in reverse-image search, facial recognition databases, etc.
- **Segmentation** — Classify EVERY SINGLE PIXEL in the image with a class label. Not just a box — exact pixel-level outlines. Used in self-driving cars (road vs pedestrian vs sky).

The key difference: classification gives one answer for the whole image. Detection gives answers per object. Segmentation gives an answer per pixel.

■ EXAM-READY ANSWER

Classification = 1 label for the whole image (e.g. 'cat').

Detection = bounding boxes + labels per object (e.g. 'person: 0.99').

Retrieval = find visually similar images in a database.

Segmentation = assign a class label to every single pixel in the image.

Q2. Why can't we just use a normal MLP (fully connected network) for images?

An MLP can only take a flat 1D list of numbers as input — called a **feature vector**. Images are 2D grids of pixels. So to use an MLP we must **flatten** the image: read all pixels out row by row (or column by column) into one long list.

Why is this bad? Pixels that are next to each other in the real image — for example, two pixels forming part of the same edge — can end up very far apart in the flattened list. The network no longer knows which pixels were neighbours. This is called **losing local connectivity / spatial relationships**. A cat ear at the top of the image and a cat paw at the bottom might need to be related for classification — flattening destroys that relationship.

The second problem — parameter explosion. In a fully connected layer, EVERY input pixel is connected to EVERY neuron with its OWN separate weight. A tiny $32 \times 32 \times 3 = 3,072$ pixel values. Just the first layer connecting to 10 output classes needs $10 \times 3,072 = 30,720$ weights. A bigger image (e.g. $224 \times 224 \times 3 = 150,528$ pixels) would need millions of weights in just one layer — impossible to train properly.

Flattening example (3×3 image):

Original grid: (0,0) (0,1) (0,2)

(1,0) (1,1) (1,2)

(2,0) (2,1) (2,2)

After row-wise flatten: (0,0),(0,1),(0,2),(1,0),(1,1),(1,2),(2,0),(2,1),(2,2)

After col-wise flatten: (0,0),(1,0),(2,0),(0,1),(1,1),(2,1),(0,2),(1,2),(2,2)

Notice: (0,0) and (2,0) are vertical neighbours in the image,
but 6 positions apart after row-wise flattening.

■ EXAM-READY ANSWER

Two reasons MLPs fail for images:

1. Flattening destroys local connectivity — pixels that were spatial neighbours become far apart in the vector, losing spatial relationships.

2. Parameter explosion — every pixel connects to every neuron with its own weight (e.g. a $32 \times 32 \times 3$ image $\rightarrow$ 30,720 weights for just one layer). This does not scale and requires unrealistic amounts of training data.

Q3. What is the Curse of Dimensionality, and why does it hurt MLPs on images?

When you have more features (more dimensions), you need **exponentially more data** to cover the possible combinations. Think of it this way: with 1 binary feature there are 2 combinations, with 2 features there are 4, with 3 features there are 8, with M features there are 2^M combinations. The more combinations exist, the harder it is for any fixed amount of training data to cover them all.

With raw pixels as features, we have thousands of dimensions. Our training data becomes relatively tiny and sparse — the model ends up just memorising examples rather than learning general patterns. This is **overfitting**. The **Hughes/peaking phenomenon** describes this exactly: adding more features actually improves performance first, but then performance drops again once you pass the ideal number (without adding more data to match).

Coverage = $2 / 2^M$ (per training example with M binary features)
M=1: 100% M=2: 50% M=3: 12.5% M=10: 0.2%
So with raw pixels (M = thousands), each example covers almost nothing.

■ EXAM-READY ANSWER

Curse of dimensionality = as the number of input features grows, the training data needed grows exponentially.

With raw pixels as features (e.g. 3,072 for a 32×32 colour image), an MLP needs an unrealistic amount of data to avoid overfitting. The model memorises instead of generalising (overfitting). This is shown by the Hughes phenomenon: performance peaks then drops as dimensionality increases without enough data.

Q4. What makes images specifically difficult to classify — the 6 challenges?

Even if the same object appears in two images, it might look very different at the pixel level because of these 6 challenges:

- **Translation** — the object is shifted left, right, up, or down in the frame.
- **Rotation** — the object is rotated or tilted at a different angle.
- **Noise** — random pixels are corrupted or the image is grainy.
- **Varying size (scale)** — the object appears larger or smaller (zoomed in/out).
- **Contour deviations** — the outline or shape of the object varies slightly.
- **Viewpoint variation** — the object is photographed from a different angle.

A good model must be **robust** to all of these — it should still predict 'cat' whether the cat is in the top-left or bottom-right of the image, rotated or not.

■ EXAM-READY ANSWER

Images are challenging because of: translation (position change), rotation (angle change), noise (pixel corruption), varying size (scale), and contour/shape deviations. A good model must be robust to all of these — i.e. predict the same class regardless.

PART 2 — Feeding an Image Into a CNN the Right Way

Q5. If flattening is bad, how does a CNN handle the image input?

Instead of squashing the image into a 1D vector, a CNN keeps the image in its **natural 2D structure** and adds the colour channels as a third dimension. This is called treating the image as a **3D volume**.

- **Width** — how many pixels across (e.g. 32)
- **Height** — how many pixels tall (e.g. 32)
- **Depth** — how many colour channels (e.g. 3 for RGB: Red, Green, Blue)

For a colour image: depth = 3 (one 2D slice per colour channel). For a greyscale image: depth = 1. By keeping the spatial structure, nearby pixels stay nearby in the data — local connectivity is preserved.

32×32 colour image = 3D volume: 32 (width) × 32 (height) × 3 (depth)
Total numbers = 32 × 32 × 3 = 3,072 (same count, but spatially organised)
Greyscale image: width × height × 1
RGB image: width × height × 3

■ EXAM-READY ANSWER

A CNN does NOT flatten the image. Instead it treats the image as a 3D volume: width × height × depth, where depth = number of colour channels (3 for RGB, 1 for greyscale). This preserves spatial/local relationships between neighbouring pixels.

PART 3 — The Convolution Operation

Q6. What is a convolution kernel (filter) and what does it do?

A **convolution kernel** (also called a filter) is a small matrix of numbers. Typical sizes are 1×1, 3×3, 5×5, 7×7, or 11×11. This kernel is **slid** (moved step by step) over the entire input image. At each position, it calculates ONE output number — this number represents how strongly a certain pattern (like an edge, a corner, or a colour gradient) is present at that location.

The key insight: the kernel weights are **trainable parameters**. They start as random numbers and are learned during training via backpropagation. The network decides for itself what pattern each filter should detect — early filters often learn to detect edges, later ones detect more complex shapes.

The result of sliding one kernel over the whole image is called a **feature map** (or activation map). It shows WHERE in the image the pattern that kernel detects is present.

■ EXAM-READY ANSWER

A convolution kernel/filter is a small trainable matrix (e.g. 3×3) of weights. It slides over the input image and computes a dot product at each position. The result is a feature map showing where a specific pattern (e.g. edge, corner, gradient) is present in the image. Kernel weights are trainable — they are learned during training, not set by hand.

Q7. What is a 'receptive field' and how is one output value computed? (the formula)

The **receptive field** is the small patch of the input image that the kernel is currently sitting on top of. It is the region that one output neuron 'sees'. The size of the receptive field equals the size of the kernel (e.g. for a 3×3 kernel, the receptive field is a 3×3 patch of pixels).

To compute one output value: multiply each kernel weight by the corresponding pixel value underneath it, then add all these products together. This is called a **dot product** (or element-wise multiply then sum).

Convolution formula:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$$

$I(i,j)$ = pixel value at position (i,j) in the input image

$K(m,n)$ = kernel weight at position (m,n)

$S(i,j)$ = output feature map value at position (i,j)

Worked example – 3×3 kernel [a,b,c / d,e,f / g,h,i] on image:

Row 0: 0 1 2 3 4 5 6

Row 1: 7 8 9 10 11 12 13

Row 2: 14 15 16 17 18 19 20

Top-left patch (0,1,2 / 7,8,9 / 14,15,16):

$$\text{Output} = 0a + 1b + 2c + 7d + 8e + 9f + 14g + 15h + 16i$$

Next patch shifted 1 right (1,2,3 / 8,9,10 / 15,16,17):

$$\text{Output} = 1a + 2b + 3c + 8d + 9e + 10f + 15g + 16h + 17i$$

■ EXAM-READY ANSWER

The receptive field = the patch of the input image the kernel is currently positioned over — the region one output neuron 'sees'.

Formula: $S(i,j) = \text{sum over } m,n \text{ of } [I(i+m, j+n) \cdot K(m,n)]$

In plain words: multiply each kernel weight by the pixel underneath it, then sum all of those products. This gives one value in the feature map.

PART 4 — Padding, Stride, and the Output-Size Formula

Q8. What is padding and why do we need it?

When the kernel slides over the image, it can be fully placed on the image at most positions. But at the **edges and corners**, the kernel would hang off the image — so without any fix, the kernel is placed fewer times on edge pixels than on centre pixels. This means edge/corner information is **under-represented** in the output. Also, every convolution shrinks the image (the output is smaller than the input).

Solution: padding. We add a border of extra pixels filled with zeros around the image before doing the convolution. The parameter P (padding) says how many zero rows/columns to add on each side. With padding $P=1$, every original pixel (including corners) gets equal treatment from the kernel, and we can also control whether the output size matches the input.

Effect of padding $P=1$ on a 3×3 image:

Padded image becomes 5×5 (zeros on all four sides):

0 0 0 0 0

0 5 2 1 0

0 3 8 6 0

0 4 7 2 0

0 0 0 0 0

Now the kernel can be centred on EVERY original pixel, including corners.

■ EXAM-READY ANSWER

Padding = adding a border of zeros around the image before convolution. Reason 1: without padding, edge/corner pixels are covered fewer times than central pixels — their information is under-used. Reason 2: padding lets us control the output size (with appropriate P, output can equal input size). P=0 = no padding (valid convolution). P=1 = one row/column of zeros added on each side.

Q9. What is stride and how does it affect the output?

Stride is the step size — how many pixels the kernel moves each time it slides to the next position. With stride 1: the kernel moves 1 pixel each step — very detailed, large output. With stride 2: the kernel skips every other position — smaller output, less computation. A larger stride = a smaller output feature map.

Example: 7×7 input, 3×3 kernel, P=0:

Stride 1 → output size = $(7-3)/1 + 1 = 5 \rightarrow 5 \times 5$ output

Stride 2 → output size = $(7-3)/2 + 1 = 3 \rightarrow 3 \times 3$ output

Stride 3 → output size = $(7-3)/3 + 1 = 2.33 \rightarrow \text{INVALID (not a whole number!)}$

■ EXAM-READY ANSWER

Stride = number of pixels the kernel shifts at each step. Stride 1 → small step, large detailed output. Stride 2 → bigger step, smaller coarser output (less computation). Not all stride values are valid — the output size must be a whole number.

Q10. What is the output-size formula and what happens when numbers don't fit?

All four factors — input size, kernel size, padding, stride — are related by one formula that tells you the exact output size after convolution. You **MUST** use this in exam calculations.

$$N_{\text{out}} = \text{floor}((N_{\text{in}} + 2P - F) / s) + 1$$

N_{in} = input size (width or height)

F = kernel/filter size (width or height)

P = padding

s = stride

N_{out} = output size

For non-square images: calculate separately for width and height.

Valid only if result is a whole number.

If not whole: either only take full steps (floor), or adjust P .

Example calculations

$N_{\text{in}}=7, F=3, P=0, s=1: (7+0-3)/1 + 1 = 5 \checkmark$

$N_{\text{in}}=7, F=3, P=0, s=2: (7+0-3)/2 + 1 = 3 \checkmark$

$N_{\text{in}}=7, F=3, P=0, s=3: (7+0-3)/3 + 1 = 2.33 \times$ (invalid!)

$N_{\text{in}}=3, F=2, P=1, s=3: (3+2-2)/3 + 1 = 2 \checkmark$ (used in mock exam!)

■ EXAM-READY ANSWER

Output-size formula: $N_{\text{out}} = (N_{\text{in}} + 2P - F) / s + 1$

If the result is not a whole number, that combination of kernel/stride/padding is invalid. Fix by adjusting padding or only taking full steps. For non-square images, apply the formula separately to width and height.

PART 5 — Parameter Sharing, Multiple Filters, and Feature Levels

Q11. How does convolution become a layer in the neural network, and how many weights does it need?

A convolution result at one position is exactly what a neuron does: multiply inputs by weights, sum them, plus a bias. So we call each convolution output value a **neuron**. All the neurons produced by sliding one kernel over the image together form a **convolutional layer**.

How many weights does one filter need? The kernel has $F \times F$ values, but the image has depth (channels), so the kernel also has depth. Total weights = $F \times F \times \text{depth}$, plus 1 for the bias. This is MUCH less than a fully connected layer.

Weights per filter = $F \times F \times \text{depth} + 1$ (the +1 is the bias)

Example: $32 \times 32 \times 3$ image, $5 \times 5 \times 3$ filter:

Weights = $5 \times 5 \times 3 + 1 = 76$

A fully connected layer connecting 3,072 inputs to just ONE neuron = 3,073 weights.
So the filter needs only 76 vs 3,073 — about 40× fewer!

IMPORTANT: 76 is the TOTAL regardless of how many positions ($28 \times 28 = 784$) the kernel is applied to. Do NOT multiply by 28×28 .

■ EXAM-READY ANSWER

Weights per convolutional filter = $F \times F \times \text{depth} + 1$ (for bias). Example: $5 \times 5 \times 3$ filter = $5 \times 5 \times 3 + 1 = 76$ weights. This is independent of how many positions the kernel slides over. Compare: a fully connected layer on the same input would need thousands of weights.

Q12. Why does parameter sharing reduce the number of weights so drastically?

When a filter detects 'a vertical edge', that pattern looks the same whether it appears in the top-left corner or bottom-right corner of the image. So instead of learning **new weights** for every position the kernel visits, the network reuses (shares) the **same kernel weights everywhere**. Only the patch of image under the kernel changes. This is called **parameter sharing**.

Why is this valid? Because of **translation invariance** — the same feature (e.g. a horizontal edge) can appear anywhere in the image, and the same filter should detect it regardless of location.

Parameter sharing — the numbers

$32 \times 32 \times 3$ image, $5 \times 5 \times 3$ filter, no padding → 28×28 output positions.

With parameter sharing: still only 76 weights per filter.

Without sharing (like a fully connected layer): $76 \times 784 = 59,584$ weights!

Reduction factor: $784 \times$ fewer parameters due to parameter sharing.

■ EXAM-READY ANSWER

Parameter sharing = the same filter weights are reused at every spatial position as the kernel slides over the image, instead of learning new weights per position. This is justified by the assumption that the same feature (e.g. an edge) looks the same wherever it appears in the image (translation invariance). Result: dramatically fewer trainable parameters.

Q13. Why do we use multiple filters per layer, and what happens as we go deeper?

One filter can only detect ONE type of pattern. To detect many useful patterns simultaneously (different orientations of edges, colours, textures, corners), we use **multiple filters** in one convolutional layer. Each filter produces its own 2D feature map. Stacking all these maps gives the output a third dimension — the output depth equals the number of filters used.

As we stack multiple convolutional layers on top of each other, the features become more and more abstract and complex:

- **Early layers (low-level)** — detect simple things like edges, colour gradients, corners.
- **Middle layers (mid-level)** — detect textures, parts of objects (eyes, wheels).
- **Deep layers (high-level)** — detect whole object-like patterns (faces, cars).

Example: $32 \times 32 \times 3$ image, 6 filters each of size $5 \times 5 \times 3$ (no padding, stride 1):

Output size per filter = $(32-5)/1 + 1 = 28$

Output volume: $28 \times 28 \times 6$ (one 28×28 map per filter)

Weights for all 6 filters: $6 \times (5 \times 5 \times 3 + 1) = 6 \times 76 = 456$

Next CONV layer (10 filters of $5 \times 5 \times 6$):

Input depth is now 6 (output of previous layer)

Output: $(28-5)/1+1 = 24 \rightarrow 24 \times 24 \times 10$

Weights: $10 \times (5 \times 5 \times 6 + 1) = 10 \times 151 = 1,510$

■ EXAM-READY ANSWER

Multiple filters per layer = each filter detects a different pattern, giving output depth = number of filters. Stacking multiple Conv+Pool layers builds feature hierarchy: early layers detect low-level features (edges), middle layers detect mid-level features (textures, parts), deep layers detect high-level features (whole objects). Weights for all filters in one layer = $(F \times F \times \text{depth} + 1) \times \text{num_filters}$.

PART 6 — Pooling Layer

Q14. What is pooling, why do we use it, and how does max-pooling work exactly?

After several convolution layers, the feature maps are still large and contain slightly redundant information. **Pooling** shrinks them by summarising small local regions into a single value. This is also called **downsampling**.

Max-pooling (the most common): slide a small window (typically 2×2) over the feature map with a stride (typically 2), and keep only the **largest value** in each window, throwing away the rest.

Why is this useful?

- **Reduces parameters** — smaller feature maps $\rightarrow$ fewer weights in later layers.
- **Prevents overfitting** — forces the network to keep only the strongest signals.
- **Robustness to small shifts/noise/scale** — if an edge moved 1 pixel, the max in that region is still activated. The network becomes translation-tolerant.

Max-pooling formula:

$P_{\text{gmax}} = \max \text{ over all } (i,j) \text{ in window of } F(i,j)$

Average-pooling formula:

$P_{\text{gavg}} = (1/MN) \times \sum F(i,j) \text{ over the window}$

Worked example – 4×4 feature map, 2×2 max-pool, stride 2:

Input: $\begin{bmatrix} 1 & 1 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 3 & 2 & 1 & 0 \\ 1 & 2 & 3 & 4 \end{bmatrix}$

Top-left {1,1,5,6} → max = 6

Top-right {2,4,7,8} → max = 8

Bottom-left {3,2,1,2} → max = 3

Bottom-right {1,0,3,4} → max = 4

Output: $\begin{bmatrix} 6 & 8 \\ 3 & 4 \end{bmatrix}$

224×224×64 with 2×2 pool stride 2 → 112×112×64 (depth unchanged!)

■ EXAM-READY ANSWER

Max-pooling = slide a window (e.g. 2×2) over the feature map and keep only the maximum value in each window, discarding the rest. Depth is unchanged. Benefits: (1) reduces number of parameters, (2) prevents overfitting, (3) increases robustness to translation, noise, and small scale changes. In practice: typically 2×2 pool with stride 2.

PART 7 — Classification Layers: Flatten → Fully Connected → Softmax

Q15. What is the full CNN architecture for classification, and what does each part do?

A CNN used as a classifier has two main sections:

- **Feature learning section** (repeated as needed): Conv + ReLU → Pooling. This section learns WHAT patterns exist in the image and WHERE they are.
- **Classification section**: Flatten → Fully Connected layer → Softmax. This section takes the learned features and outputs class probabilities.

Why does the Conv+Pool block repeat? Because one block only finds simple patterns. Repeating it lets the network build more and more abstract features (edges → textures → objects).

Full CNN classifier pipeline:

INPUT (W×H×D)

- [CONV + ReLU] ← feature detection (trainable kernel weights)
- [POOL] ← downsampling, reduce parameters, prevent overfitting
- [CONV + ReLU] ← more abstract features
- [POOL] ← downsampling again
- ... (repeat as needed)
- FLATTEN ← reshape 3D volume to 1D vector
- FULLY CONNECTED ← learns classification from features (K outputs)
- SOFTMAX ← turns raw numbers into probabilities

OUTPUT: class probabilities

For REGRESSION: same but no Softmax — just a final FC layer with raw numbers.

■ EXAM-READY ANSWER

CNN classifier layers: INPUT → [CONV+ReLU → POOL] × n → FLATTEN → FC → SOFTMAX.

Feature learning layers = CONV+ReLU and POOL (these repeat). These perform feature extraction — they learn patterns from the image. Conv+Pool repeats because stacking layers allows progressively more abstract features (low-level → mid-level → high-level). For regression: no Softmax, just a final FC layer.

Q16. What does flattening do and what does the fully connected layer output?

After the last pooling layer, we have a 3D volume of learned features (e.g. 7×7×64). This cannot go directly into a standard fully connected layer, which needs a 1D vector. So we **flatten** it: reshape the 3D volume into a 1D vector by simply reading out all values in order (e.g. 7×7×64 = 3,136 values in a single vector).

The **fully connected (FC) layer** is then a normal MLP layer: every neuron connects to every value in the flattened vector with its own weight. For a classifier with K classes, the FC layer outputs K numbers (one per class). These are called **logits** — raw scores, not yet probabilities.

■ EXAM-READY ANSWER

Flatten = reshape the final 3D feature volume into a 1D vector (e.g. 7×7×64 → 3,136 values).

This converts the spatial feature map into the format an MLP/FC layer expects. The FC layer outputs K numbers (logits) where K = number of classes. For regression tasks, this FC output IS the final answer (no Softmax needed).

Q17. What is Softmax and why do we use it instead of just picking the max logit?

The raw FC outputs (logits) are just numbers — they can be positive or negative, any size. To get actual class **probabilities** we use Softmax. Softmax squashes all logits into values between 0 and 1 that sum to exactly 1.

Why not just argmax (pick the biggest number)? Because argmax is not differentiable — we cannot use it to compute gradients for training. Also, argmax throws away confidence information (90% vs 55% both just give 'class 1'). Softmax gives us a proper probability distribution we can use with cross-entropy loss for training.

Softmax formula:

$\sigma(z)_i = \exp(z_i) / \sum_j \exp(z_j)$ for $j = 1$ to K

z = vector of logits (raw FC output)

K = number of classes

$\exp = e^x$ (exponential function)

Worked example:

Logits: [1.3, 5.1, 2.2, 0.7, 1.1]

After softmax: [0.02, 0.90, 0.05, 0.01, 0.02]

Sum = 0.02+0.90+0.05+0.01+0.02 = 1.00 ✓

Prediction = Class 2 (probability 90%)

■ EXAM-READY ANSWER

Softmax converts raw logits into a probability distribution: all outputs are positive and sum to 1. Formula: $\sigma(z)_i = \exp(z_i) / \sum_j \exp(z_j)$. We use Softmax instead of argmax because: (1) Softmax is differentiable — needed for backpropagation/training. (2) It provides a meaningful probability distribution — confidence/certainty is preserved. (3) It is required for cross-entropy loss computation during training.

PART 8 — Calculation Practice (Mock Exam Style)

Q18. MOCK EXAM Task 4b: Apply filter K to input I with $P=1$, $s=3$. Use ReLU. (9 points)

This is the biggest calculation question from the mock exam. Always follow this exact order: (1) compute output size, (2) pad the image, (3) compute each output value, (4) apply ReLU last.

Given:

$I = \begin{bmatrix} 5 & 2 & 1 \\ 3 & 8 & 6 \\ 4 & 7 & 2 \end{bmatrix}$, $K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$

Padding $P=1$, Stride $s=3$

STEP 1 – Output size:

$$N_{out} = (N_{in} + 2P - F) / s + 1$$
$$= (3 + 2 \times 1 - 2) / 3 + 1$$
$$= (3 + 2 - 2) / 3 + 1$$
$$= 3/3 + 1$$
$$= 1 + 1 = 2$$

Output is 2×2 .

STEP 2 – Pad the image ($P=1$, add 1 row/col of zeros on each side):

Padded image (5×5):

Row0: 0 0 0 0 0
Row1: 0 5 2 1 0
Row2: 0 3 8 6 0
Row3: 0 4 7 2 0
Row4: 0 0 0 0 0

STEP 3 – Slide kernel with stride 3:

Kernel positions (top-left corner of 2×2 kernel):

(0,0), (0,3), (3,0), (3,3)

Output(0,0): patch = $\begin{bmatrix} 0 & 0 \\ 0 & 5 \end{bmatrix}$
 $= 0 \times 1 + 0 \times 0 + 0 \times (-1) + 5 \times 1 = 5$

Output(0,1): patch = $\begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$
 $= 0 \times 1 + 0 \times 0 + 1 \times (-1) + 0 \times 1 = -1$

Output(1,0): patch = $\begin{bmatrix} 0 & 4 \\ 0 & 0 \end{bmatrix}$
 $= 0 \times 1 + 4 \times 0 + 0 \times (-1) + 0 \times 1 = 0$

Output(1,1): patch = $\begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$
 $= 2 \times 1 + 0 \times 0 + 0 \times (-1) + 0 \times 1 = 2$

Raw feature map: $\begin{bmatrix} 5 & -1 \\ 0 & 2 \end{bmatrix}$

STEP 4 – Apply ReLU: $f(x) = \max(0, x)$

$\begin{bmatrix} 5 & -1 \\ 0 & 2 \end{bmatrix} \rightarrow \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$

FINAL ANSWER: $\begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$

Common mistakes to avoid

1. Forgetting to pad BEFORE computing the output size.
2. Using $N_{in}=3$ in the formula but computing patches on the UNpadded image.
3. Applying ReLU BEFORE the convolution — always apply ReLU AFTER.
4. Getting the kernel element order wrong: $K[0,0]$ goes to top-left, $K[1,1]$ to bottom-right.
5. Computing output size wrong: don't forget the +1 at the end.

■ EXAM-READY ANSWER

Always follow this order: 1) output size formula, 2) pad the image, 3) slide kernel and compute dot products, 4) apply ReLU (negatives $\rightarrow$ 0). Show each step clearly for full marks.

Q19. MOCK EXAM Task 4c: Max-pooling on a grid — read the answer and define it.

The mock exam gives a feature map grid with one 2×2 window marked with a dashed border. You need to: (1) find the maximum value inside the dashed window, (2) write a one-sentence definition of max-pooling.

Given feature map:

```
[ 1 -4 | 5 7 ]  
[ 2 0 | 1 1 ] ← dashed window = top-left 2x2 = {1, -4, 2, 0}  
[ 3 9 9 9 ]  
[ 0 0 -2 0 ]
```

Dashed window contains: 1, -4, 2, 0

Max-pool result = $\max(1, -4, 2, 0) = 2$

■ EXAM-READY ANSWER

Max-pool result for the marked 2×2 window {1, -4, 2, 0} = 2.

Definition: Max-pooling takes a window (e.g. 2×2) over a feature map and retains only the largest value in that window, discarding all others, thereby reducing spatial dimensions while preserving the strongest activations.

Q20. How many trainable weights does a conv layer have with multiple filters?

Weights for one layer with N filters:

Total = $N \times (F \times F \times \text{depth} + 1)$

Example: $32 \times 32 \times 3$ input, 6 filters of size 5×5 :

Weights per filter = $5 \times 5 \times 3 + 1 = 76$

Total for 6 filters = $6 \times 76 = 456$

Next layer (input depth = 6, 10 filters of 5×5):

Weights per filter = $5 \times 5 \times 6 + 1 = 151$

Total = $10 \times 151 = 1,510$

■ EXAM-READY ANSWER

Total weights in a conv layer = $(F \times F \times \text{depth} + 1) \times \text{number_of_filters}$. The '+1' is for the bias of each filter. Depth for the FIRST conv layer = number of input channels. Depth for later conv layers = number of filters in the PREVIOUS layer.

PART 9 — Other Likely Exam Questions

Q21. What is the ReLU activation function and why is it used after convolution?

ReLU stands for **Rectified Linear Unit**. It is applied element-wise after the convolution to add non-linearity.

ReLU: $f(x) = \max(0, x)$

$x = 5 \rightarrow f(5) = 5$ (positive values pass through unchanged)

$x = -3 \rightarrow f(-3) = 0$ (negative values are set to zero)

$x = 0 \rightarrow f(0) = 0$

Why ReLU?

- Fast to compute — just a max operation.
- Introduces non-linearity — without this, stacking layers would just be one big linear operation.
- Kills negative values — makes the network focus on positive (relevant) activations.
- Avoids vanishing gradient better than sigmoid/tanh for deep networks.

■ EXAM-READY ANSWER

ReLU: $f(x) = \max(0, x)$ — keeps positive values unchanged, sets negatives to zero. Used after convolution to add non-linearity (without non-linearity, deep networks reduce to a single linear transform). Fast to compute. Helps avoid vanishing gradient in deep networks.

Q22. Fill in the MLP vs CNN comparison table.

Criterion	MLP	CNN
Input format	1D flat vector (requires flattening)	3D volume (width × height × depth)
Spatial relationships	Lost (destroyed by flattening)	Preserved (local connectivity)
Parameters for large images	Very high (parameter explosion)	Low (parameter sharing)
Robust to translation	No	Yes (with pooling)
Robust to rotation/noise	No	Partially (with data augmentation + pooling)
Manual feature engineering	Needed (or raw pixels)	Not needed — features learned automatically
Overfitting risk on images	High	Lower (fewer params, pooling helps)
Typical use for images	Not recommended	State of the art

Q23. Can a CNN be used for regression (not just classification)? What changes?

Yes. The feature-learning section (Conv+ReLU+Pool layers) is identical. Only the final output layer changes:

- **Classification:** final layer = Fully Connected (K outputs) + Softmax → class probabilities.
- **Regression:** final layer = Fully Connected (1 or more outputs, NO Softmax) → raw numbers.

■ EXAM-READY ANSWER

A CNN can do regression OR classification. The Conv+Pool feature-learning layers are the same in both cases. For classification: add FC + Softmax at the end (outputs probabilities). For regression: add only FC at the end (outputs raw numerical values, no Softmax).

Q24. Task 4a type: What layers does a CNN classifier have, which do feature extraction, and why repeat?

This is one of the most common 6-point questions in the mock exam. Cover all three parts.

■ EXAM-READY ANSWER

A CNN classifier typically consists of:

Feature extraction layers: Convolution + ReLU (activation) and Pooling layers.

Classification layers: Flatten, Fully Connected (FC), Softmax.

Convolutional layers are used for feature extraction because they apply learnable filters (kernels) that detect spatial patterns (edges, textures, shapes) in the image. ReLU adds non-linearity. Pooling reduces spatial dimensions and prevents overfitting.

It is useful to have multiple Conv+Pool blocks because each block learns progressively more abstract features: early blocks detect low-level features (edges, gradients), later blocks detect mid- and high-level features (object parts, whole objects). Stacking layers allows the network to build complex representations from simple ones.

PART 10 — Final Cheat Sheet (Everything on One Page)

Concept	Key Fact / Formula
4 CV tasks	Classification (1 label), Detection (boxes), Retrieval (similar images), Segmentation (per pixel)
MLP problem 1	Flattening destroys local connectivity — spatial neighbours become distant
MLP problem 2	Parameter explosion — every pixel → every neuron, own weight
Curse of dimensionality	Training data needed grows exponentially with features; coverage = $2/2^M$ %
Image challenges	Translation, rotation, noise, size change, contour deviation
CNN input	3D volume: width × height × depth (depth = channels, e.g. 3 for RGB)
Kernel/filter	Small trainable matrix (e.g. 3×3); slides over image; learns patterns
Convolution formula	$S(i,j) = \sum_m \sum_n I(i+m,j+n) \cdot K(m,n)$ (dot product per position)
Feature map	Output of sliding one kernel over the image
Receptive field	The image patch one output neuron 'sees' (= kernel size)
Padding P	Adds zero border; fixes edge pixels; controls output size
Stride s	Step size of kernel; larger stride = smaller output
Output size	$N_{out} = (N_{in} + 2P - F)/s + 1$ (must be whole number, per dimension)
Weights per filter	$F \times F \times \text{depth} + 1$ (bias) — does NOT depend on number of positions
Parameter sharing	Same filter weights reused at every position → huge param reduction
Multiple filters	Each filter detects a different pattern; output depth = num filters
Feature hierarchy	Early layers: low-level (edges); deep layers: high-level (objects)
Max-pooling	Slide 2×2 window, keep max value; reduces size, prevents overfitting
Pool benefits	Fewer params, less overfitting, robust to shift/noise/scale
ReLU	$f(x) = \max(0, x)$: keeps positives, zeros negatives, adds non-linearity
Flatten	Reshape final 3D feature volume into 1D vector for FC layer
FC layer	Fully connected; outputs K numbers (K = num classes) = logits
Softmax	$\sigma(z)_i = \exp(z_i) / \sum_j \exp(z_j)$; turns logits into probabilities (sum=1)

CNN classifier pipeline	INPUT → [CONV+ReLU → POOL]×n → FLATTEN → FC → SOFTMAX
CNN for regression	Same but NO Softmax; final FC outputs raw numbers
Calculation order	1) output size 2) pad image 3) slide+dot product 4) apply ReLU